

EXPERIMENT NO. 3

Sum of Squares of Distinct Counts of All Subarrays

Programming Language: C

Name : Bodham susma

Register No. : 192571065

Aim

To write a C program that computes the sum of the squares of the distinct counts of every contiguous subarray of an array.

Question

You are given a 0-indexed integer array `nums`. The distinct count of a subarray of `nums` is defined as: Let `nums[i..j]` be a subarray of `nums` consisting of all the indices from `i` to `j` such that $0 \leq i \leq j < \text{nums.length}$. Then the number of distinct values in `nums[i..j]` is called the distinct count of `nums[i..j]`. Return the sum of the squares of distinct counts of all subarrays of `nums`. A subarray is a contiguous non-empty sequence of elements within an array.

Example 1

Input: `nums = [1,2,1]`

Output: 15

Explanation: Six possible subarrays are:

- `[1]`: 1 distinct value
- `[2]`: 1 distinct value
- `[1]`: 1 distinct value
- `[1,2]`: 2 distinct values
- `[2,1]`: 2 distinct values
- `[1,2,1]`: 2 distinct values

The sum of the squares of the distinct counts in all subarrays is equal to $1^2 + 1^2 + 1^2 + 2^2 + 2^2 + 2^2 = 15$.

Example 2

Input: `nums = [1,1]`

Output: 3

Explanation: Three possible subarrays are:

- `[1]`: 1 distinct value
- `[1]`: 1 distinct value
- `[1,1]`: 1 distinct value

The sum of the squares of the distinct counts in all subarrays is equal to $1^2 + 1^2 + 1^2 = 3$.

Algorithm

Step 1: Start.

Step 2: Store the array `nums`, compute its size `n`, and set `totalSumOfSquares = 0`.

Step 3: For each starting index `i` from 0 to `n - 1`, create an empty array `uniqueElements` and set `distinctCount = 0`.

Step 4: For each ending index `j` from `i` to `n - 1`, take `currentNum = nums[j]` and check whether it already exists in `uniqueElements`.

Step 5: If it does not exist, add it to `uniqueElements` and increment `distinctCount`.

Step 6: Add `distinctCount * distinctCount` to `totalSumOfSquares`.

Step 7: After all subarrays are processed, print `totalSumOfSquares`.

Step 8: Stop.

Program (C)

```
#include <stdio.h>

int main() {
    // Example 1 Input Array (Change this to test other cases)
    int nums[] = {1, 2, 1};
    int n = sizeof(nums) / sizeof(nums[0]);

    int totalSumOfSquares = 0;

    // Outer loop: starting point of the subarray
    for (int i = 0; i < n; i++) {

        // We will keep track of unique numbers seen in the current subarray starting at 'i'
        // Let's create a temporary array to store unique elements as we expand
        int uniqueElements[100]; // Assuming maximum array size is 100 for simplicity
        int distinctCount = 0;

        // Inner loop: ending point of the subarray
        for (int j = i; j < n; j++) {
            int currentNum = nums[j];
            int alreadyExists = 0;

            // Check if currentNum has already been added to our unique list for this subarray
            for (int k = 0; k < distinctCount; k++) {
                if (uniqueElements[k] == currentNum) {
                    alreadyExists = 1;
                    break;
                }
            }

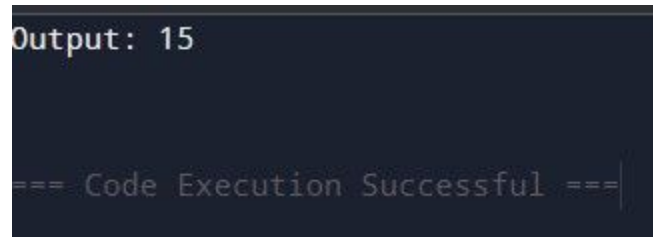
            // If it's a new unique number, add it to our list
            if (alreadyExists == 0) {
                uniqueElements[distinctCount] = currentNum;
                distinctCount++;
            }

            // Add the square of the distinct count of the current subarray nums[i..j] to our total
            totalSumOfSquares += (distinctCount * distinctCount);
        }
    }

    // Output the final result
    printf("Output: %d\n", totalSumOfSquares);

    return 0;
}
```

Output



```
Output: 15

=== Code Execution Successful ===
```

Complexity Analysis

Time Complexity: $O(n^2)$

Space Complexity: $O(1)$

Result

Thus the program was executed successfully and the output was verified.